Design Document - Biquadris

Shraddha Aangiras, Ruby Zhou, Naunidh Singh

Introduction

Biquadris is a competitive two-player falling-block game inspired by Tetris, played on parallel 18×11 boards. (Each board is 11 columns wide and 15 rows high, with 3 extra rows on the top to give room for blocks to rotate.) Players alternate turns, moving and rotating blocks to clear lines, score points, and avoid a game-over state, with additional mechanics such as blind, heavy, forced block types, and Level 4's StarBlock. This design document outlines the final structure of our program, including block mechanics, board and scoring logic, level-based generation, command parsing, and text/graphical displays. We also highlight key design decisions, differences from the initial UML, and the object-oriented principles that shaped the system's extensibility.

Overview

The structure of our Biquadris project follows an object-oriented model that separates game logic, player interaction, and display. At the center is the **Game** class, which manages turn-taking, command processing, and restarts, while delegating all gameplay rules to the active **Player**. Input is parsed through the **CommandInterpreter**, allowing flexible commands via prefix matching, multipliers, aliases, and macros.

Each **Player** owns its own **Board** and **Level**, allowing players to operate independently with different block sequences and level-specific behaviours. The **Board** manages the 18×11 grid, block placement and movement, line clearing, scoring, and game-over detection. **Block** subclasses define individual shapes through polymorphism, while shared movement and rotation logic lives in the **abstract Block base clas**s. **Levels 0–4** use the **Strategy** pattern to generate blocks using scripted sequences or weighted randomness.

Rendering is handled through two interchangeable views: **TextDisplay** for ASCII output and **GraphicsDisplay**, which uses the **Xwindow** class for graphical rendering. Both displays observe the **Board** through the **Observer** pattern and update automatically on state changes. Memory throughout the project is managed with RAII and smart pointers to ensure safe ownership and avoid leaks. The following sections detail each component and the design choices behind them.

Design

Class: Game

The **Game** class coordinates the interactions between the two players, their boards, and both display types. It manages the turn order, parses commands through the **CommandInterpreter**, initializes **Levels**, handles restarts, and runs the main game loop. Design decisions include:

- Delegates gameplay rules entirely to **Player** and **Board**, keeping **Game** focused on orchestration.
- Uses **CommandInterpreter** to separate input parsing from execution.
- Manages the lifecycle of **TextDisplay** and **GraphicsDisplay**, attaching them as observers to the **Boards**.
- Supports text-only mode and script-file initialization for deterministic testing.

Changes since DD1: Originally **Game** directly manipulated board logic, but this was refactored so **Player** owns and manages **Board** and **Level**, reducing coupling. In addition to reducing coupling, we also adopted a more explicit MVC-inspired structure. Game now acts purely as the "controller" and entry point for command execution, while Player and Board form the "model," and TextDisplay/GraphicsDisplay form the "views."

Class: CommandInterpreter

CommandInterpreter parses raw user input into canonical commands and multipliers. This separates user interaction logic from both **Game** and **Player**. Techniques used:

- Shortest-prefix matching to allow flexible input (ri -> right if unambiguous).
- Numeric multipliers (3left) extracted using string parsing.
- Alias support for user-defined command names.
- Macro expansion to treat custom sequences as a single command.

Changes since DD1: Macros and alias tables were added for extensibility, improving the interpreter's flexibility without modifying **Game**. These improvements were to avoid code duplication in Game and Player, and supporting flexible user-defined input without hardcoding new commands. Using alias tables and macro expansion allowed us to treat higher-level user input as pure strings, while the rest of the system only deals with canonical commands, a division that greatly reduced inter-class dependency.

Class: Player

The **Player** class manages all state related to a single player, including their **Board**, **Level**, score, effects, and current/next blocks. It acts as the "controller" for all actions that mutate game state. Design techniques:

- Owns unique_ptr<Board> and unique_ptr<Level> to clearly express ownership.
- Executes canonical commands such as movement, rotation, drop, level changes, and special actions.
- Implements special behaviors (blind, heavy stacking, forced block type) locally, without modifying **Board**.
- Delegates block placement and scoring to **Board** through a clean interface.

Changes since DD1: **Player** absorbed responsibility for special effects to simplify **Game**, improving cohesion. This shift also enabled a more disciplined application of single-responsibility principles. Player now fully encapsulates everything including heavy stacking, blind toggling, forced-block actions, and level management.

Class: Board

Board is the central class that manages the 18×11 grid, falling blocks, scoring, and game-over logic. Key techniques:

- Stores grid as vector<vector<Cell>>
- Implements full movement logic (left, right, down, rotation) using a "remove → modify → validate → place" pattern to prevent illegal state mutation.
- Tracks block IDs in activeBlocks_ to calculate full-block removal bonuses.
- Clears rows by shifting grid rows downward and recalculating scoring.
- Stores blind effect state via blindActive_ flag, which displays check during rendering.
- Notifies all displays via the Observer pattern after each state change.
- Tracks **heavy state** transferred from Player to correctly apply a one-cell downward push before any other move

Changes since DD1: Block ID tracking was added. We moved effect coordination into Player (e.g., applyBlind(), applyHeavy()), while Board stores minimal effect state (e.g., blindActive_) for rendering decisions. This increased cohesion by separating effect triggers (Player) from effect rendering (Board/Displays).

Class: Cell

Represents a single grid square on the **Board**. Design features:

- Stores position, occupancy, block type, and block ID.
- Provides simple set and clear methods.
- Has no dependencies on other components to maintain high cohesion.

Changes since DD1: Added blockId_ to support complete block removal scoring.

Class: Block (Abstract Base Class)

Block defines the shared behavior for all block types, including position tracking, movement, and rotation. Techniques used:

- Polymorphism allows **Board** to treat all block types uniformly.
- Coordinates stored as a vector of (row, col) pairs.
- Implements rotation using bounding-box math.
- Stores levelGenerated and id for scoring and tracking.

Changes since DD1: Block objects track a unique block ID, levelGenerated, and full coordinate lists, enabling the scoring system that awards bonuses for clearing entire blocks. This required restructuring both Block and Cell compared to the original UML. We also unified movement logic into the base Block class so subclasses only define initial geometry.

Subclasses: IBlock, JBlock, LBlock, OBlock, SBlock, ZBlock, TBlock, StarBlock

Each subclass defines its initial geometry using addCell() calls. Design features:

- Inherit all movement and rotation logic from **Block**.
- StarBlock overrides construction to support Level 4 placement rules.
- Open for extension: adding new block types requires only a new subclass.

Class: Level (Abstract Base Class) and Subclasses (Level0–4)

Level encapsulates block-generation strategies using the Strategy pattern. Techniques used:

- generateBlock() implemented differently in each **Level** for probabilities or sequence files.
- isHeavy(), setRandom(), and setSequenceFile() allow customization of difficulty behavior.
- Level4 overrides onRowCleared() to manage StarBlock counters.

Class: createLevel() Factory

Provides a single point for creating Level objects. Design techniques:

- Uses a factory to map level numbers to concrete types.
- Isolated from Player logic, supporting easy extension for new Levels.

Class: Subject and Observer

Implement the Observer pattern used by Board and the displays. Design techniques:

- Subject stores raw Observer pointers but does not own them.
- notifyObservers() pushes board updates to both **TextDisplay** and **GraphicsDisplay**.
- Allows new display types without modifying Board.

Class: TextDisplay

Renders an ASCII version of the board for terminal output. Techniques used:

- Stores a 2D buffer updated on each notification.

Provides next-block preview using **Board's** public interface.

Class: GraphicsDisplay

Provides graphical rendering using Xwindow. Design features:

Observes both boards simultaneously, drawing them side-by-side.
Uses color coding for different block types.
Renders level, score, and next-block preview text.
Handles blind effect by drawing masked regions.

Class: Xwindow

A thin abstraction layer over X11.

Provides drawing primitives: rectangles, text, and color management.
Encapsulates connection to the low-level display server.

Overall Techniques Used to Address Design Challenges

Encapsulation: Classes expose only the interfaces needed for their roles. Board exposes its grid as a const reference, preventing external modification while allowing displays to render state, and **Block** subclasses cannot modify unrelated state, keeping responsibilities clearly separated.

Polymorphism: The **Block** and **Level** hierarchies use polymorphism to support multiple block shapes and difficulty behaviors behind common interfaces. **TextDisplay** and **GraphicsDisplay** both observe the **Board** through the same **Observer** interface, allowing interchangeable rendering.

Separation of Concerns: Game manages turn flow, **Board** handles state and scoring, **CommandInterpreter** parses input, and displays render output. This division reduces coupling and makes components easier to maintain.

Safe Move Validation: Board uses a remove–modify–validate–place pattern to prevent illegal moves from corrupting state, ensuring consistent behavior without complex rollback logic.

Ease of Testing: TextDisplay and Level 0's deterministic sequences enabled straightforward unit testing, allowing core mechanics to be validated before integrating graphics.

Resilience to Change

Our Biquadris design is structured to support specification changes with minimal modification through clear abstraction boundaries and the use of core OOP patterns. The system is modular: **Game** handles orchestration, **Player** manages player-specific state, **Board** enforces gameplay rules, **Level** controls block generation, and displays render output. Because each component has a single responsibility and interacts only through well-defined interfaces, changes remain localized and do not cascade.

The **Strategy** pattern in the **Level** hierarchy isolates all difficulty logic. Since **Player** holds a unique_ptr<Level> to the abstract interface, adding a new level mode requires only creating a new subclass and updating the factory. The **Block** hierarchy offers similar extensibility: all block types inherit shared movement and rotation logic, so introducing a new shape or special block requires only defining a

new subclass without modifying **Board** or **Player**. This design maintains low coupling and high cohesion across gameplay components.

Displays are decoupled through the **Observer** pattern. **Board** notifies **TextDisplay** and **GraphicsDisplay** of updates, allowing new display types to be added without touching **Board** logic. Input parsing is isolated in **CommandInterpreter**, so new commands, aliases, or macros can be introduced without affecting gameplay logic in **Game** or **Player**.

Finally, memory management is resilient due to consistent use of RAII and smart pointers, making component replacement safe and preventing leaks. Overall, the combination of encapsulation, polymorphism, Strategy, Observer, and separated responsibilities ensures the architecture can accommodate new blocks, levels, commands, scoring rules, or display modes with small, localized changes.

Questions

Q1: Block

To support blocks that disappear if they haven't been cleared after 10 drops, we extend the model with a lightweight "aging" system. The Board (or Player) tracks a drops counter that increments on every successful drop. Blocks that can expire store two extra fields: a timed flag and a spawnDrop number recording the drop count when they were created. After every drop, the board checks its active blocks; any timed block where currentDrops - spawnDrop >= 10 is removed from the grid. Only advanced Level subclasses mark their generated blocks as timed, so lower levels behave exactly as before. This fits cleanly with the existing Level strategy and active-block tracking with minimal intrusion.

Q2: Next Block

To allow new levels to be added easily, all level-specific behaviour is hidden behind the abstract Level interface, which defines operations like block generation, heavy behaviour, and randomness control. Player and Game store only a pointer to this interface and call its methods without knowing the concrete type. Introducing a new level is simply a matter of defining a new subclass (e.g., Level5) and registering it inside a small creation factory or switch. No other component depends on the concrete classes, so only the new file and the factory need recompilation; Player, Board, and Game remain untouched.

Q3: Special Actions

To allow multiple effects to apply at the same time, and to support new effects in the future, we can model them as separate objects instead of a fixed set of boolean flags. We would introduce an abstract Effect type with methods like applyOnMove(Player&, Board&), applyOnDisplay(Board&, Display&), and bool isExpired(). Each Player maintains a std::vector<std::unique_ptr<Effect>> of active effects. At key points in the game loop (for example, after a command or at the start of a turn), the game iterates over this list and lets each effect modify behaviour: a BlindEffect masks part of the display, a HeavyEffect requests extra downward moves, a ForceEffect replaces the current block type, and so on. Expired effects remove themselves from the list. New effects are added by defining new Effect subclasses, without needing a giant if/else chain for every possible combination; composition of behaviours comes naturally from iterating the list.

Q4: Command Interpreter

To make adding or renaming commands easy, we let the CommandInterpreter own a registry that maps each canonical command name to a function that operates on the Game or Player (for example, a std::unordered_map<std::string, std::function<void(int)>>). When the user types something like 3lef, the interpreter first peels off the numeric prefix (3), then uses shortest-prefix matching on the registry to resolve lef to the full command name (left). It then just calls the stored function with the multiplier.

Renames are handled by a separate alias table that maps user-defined names (cc) to canonical names (counterclockwise), so none of the game logic has to change when a command is renamed.

On top of this, we can support a small macro language by storing macro names mapped to sequences of command strings; invoking a macro simply pushes its sequence into an internal input buffer and runs it through the same parsing pipeline as normal input. Because all of this logic lives inside CommandInterpreter, adding new commands, aliases, or macros mostly means updating these tables and the prefix resolution, while the rest of the system remains untouched.

Extra Credit Feature

Ghost Block Preview: Board supports an optional ghost mode that displays a gray shadow showing the final resting position of the current block. We solved the challenge of self-collision with a non-mutating calculation loop: the getGhostCells() function temporarily removes the active block from the grid, calculates drop position, and then returns the coordinates without modifying the board state or the active block's position. Can be toggled on and off with command “ghost”

STL Containers Used: all memory management is handled via STL containers such as vectors and smart pointers

Final Questions

1. What lessons did this project teach you about developing software in teams? If you worked alone, what lessons did you learn about writing large programs?

This project taught us how essential clear communication, consistent design decisions, and well-defined interfaces are when working in a team. We learned that even small misunderstandings, whether about ownership rules, how Blocks should interact with the Board, or when certain methods are supposed to run, can snowball into major refactors once multiple components depend on one another. Aligning on the UML early and continuously revisiting it as a shared reference point made a huge difference in keeping everyone synchronized. We also discovered how valuable early, incremental testing is; catching issues in block movement, rotation logic, notifications, and level behaviour one piece at a time saved us from extensive debugging once the full game loop, displays, and two-player interactions were wired together. We also learned how helpful it was to use Git consistently from the start because it made collaboration, merging, and reviewing changes much easier. We realized how often we needed to touch base after completing a piece of work, so we could reassign the next tasks. We also learned how to integrate code written by different people with different styles by following good practices of coding, which helped us keep the project readable. Overall, the experience reinforced that successful software teamwork depends not just on coding skill but on organization, predictability, and constant coordination to avoid surprises and keep the project cohesive.

2. What would you have done differently if you had the chance to start over?

If we could restart the project, we would take a more disciplined approach to the early phases of design.

Fully finalize the architecture before coding: We would spend more time validating the class relationships, ownership rules, and major patterns (like Observer and Strategy) up front. A large portion of our time was spent revisiting earlier assumptions and refactoring mid-implementation, which could have been avoided with a firmer initial design.

Introduce testing much earlier: We would write small, targeted tests for the Board, Block system, and Level logic before integrating the displays or player interactions.

Many bugs only showed up once multiple modules interacted, so earlier unit-style testing would have caught problems sooner and made debugging significantly easier.

Plan collaboration and workflow more intentionally: We would more clearly divide responsibilities at the start, keep branches narrowly scoped, and merge more frequently to reduce integration conflicts. Smaller, more frequent PRs would have made the codebase easier to review, prevented large merge collisions, and helped keep everyone aligned.

Overall, the biggest improvement would be committing to a stronger design and testing foundation early so that the later stages of development could move faster and with fewer surprises.